

Blind Model Checking of Workflows in Agentic AI with Zero-knowledge Proof[★]

Subhasis Thakur^{a,*,1}

^aUniversity of Galway, Galway, Ireland

ARTICLE INFO

Keywords:

Agentic AI
Model Checking
Zero knowledge proof
Workflows

ABSTRACT

An AI agent's workflow in response to a task given by the end user in natural language may lead to undesirable outcomes due to probabilistic error from its large language model. Hence, it is necessary to ensure the workflow to be executed by an agent is safe. In many cases, the end user may not have access to the internal model of the agent and hence its intended workflow. In such cases, it is necessary to perform verification of the safety of an agent's workflow without the knowledge of the workflow. In this paper, we address this problem using a zero-knowledge proof for model checking the agent's workflow where the verifier (the end user) does not have access to the workflow. We have developed such verification protocols to check whether the workflow leads to undesirable states or cycles. We prove that these verification protocols are correct and sound. We also present an experimental evaluation to estimate the time it takes to perform such verification.

1. Introduction

AI agents can autonomously execute tasks on behalf of its end user. AI agents depend on their large language model (LLM) to comprehend the end user's instruction represented in natural language and generate the desired actions to respond to such instructions. There are mainly two approaches to use AI agents to execute an end user's task: (a) Retrieval Augmented Generation (in this approach, the agent is equipped with contextual information, which it passes to its LLM along with the query from the end user) and (b) Finetuning (in this approach, the agent locally re-trains the LLM with private contextual information, and it generates its actions based on the revised LLM's response). In both approaches, the probabilistic error of the LLM poses a significant challenge to the reliability of the AI agent. This problem is further complicated when the AI agent works as a black box where the end user does not have access to the internal data of the AI agent. For example, if the end user is not part of the organization that controls and owns the AI agent, then it can not access the internal data of the AI agent.

In this paper, we address the problem of verifying a black box AI agent in terms of the correctness of the set of actions, the agent wants to execute in response to the instruction from the end user. We represent the set of actions that the agent wants to execute as a process model. We will use a Kripke structure to represent such a process model. We have developed methods to verify the correctness of such a black box process model to be used by the agent (i.e., it does not share the process model with the end user). Our approach to verify the blackbox process model of an agent is as follows:

- The end user sends a task description to the agent in natural language.

- The agent uses its LLM to generate a process model (via Kripke structure) and sends a commitment to this process model that does not reveal the process model, but it can be used to verify if the process model is correct or not.
- The agent generates a zero-knowledge proof on the Kripke structure to prove that certain undesirable states can not be reached from a given start node of the Kripke structure.
- The agent generates a zero-knowledge proof that the Kripke structure is live as we can not reach a strongly connected component (such as a cycle) from a given start node of the Kripke structure.
- The agent generates a zero-knowledge proof that the Kripke structure is fair as we can not reach a strongly connected component with a certain node label from a given start node of the Kripke structure.

The paper is organized as follows: in Section 2, we describe the blind workflow verification problem, in Section 3, we describe the proposed verification of workflows, in Section 4, we present an analysis of the correctness of the verification process, in Section 5, we present an experimental evaluation of the proposed verification method, in section 6, we discuss related literature, and we Conclude the paper in section 7.

2. Workflow Verification Problem

Let there be two parties: an end user and an AI agent, where the end user sends a task description to the agent and wants to verify if the agent intends to follow a correct procedure to execute the task. The agent generates a Kripke structure to represent its intended plan to execute the task. The agent does not share the information on the Kripke structure with the end user, as it generates a zero-knowledge

★

 subhasis.thakur@universityofgalway.ie (S. Thakur)

 (S. Thakur)

ORCID(s):

1

proof on the correctness and safety of the Kripke structure via model checking the Kripke structure.

Definition 1. A kripke structure $KS = (S, R, L, I)$ has four elements:

- S be the set of states representing a step or task in the workflow.
- R be the set of relations over the states that defines how the system moves from one state to another.
- L be a set of labels to the states. L can be a set of atomic propositions, i.e., a set of statements that are true if the system reaches a state.
- I be the set of initial states.

Definition 2. A kripke structure $KS = (S, R, L, I)$ has the following reachability problems:

- To verify the safety of the workflow described as the Kripke structure, one need to verify that certain states $S' \subset S$ can not be reached from the initial state I .
- To verify target state $t \in S$ reachability, one needs to check if there is at least one path from the initial states to the target state.

Definition 3. In a kripke structure $KS = (S, R, L, I)$ a strongly connected component is a subgraph KS' of KS where each state in $S_1 \in KS'$ is reachable from another state in KS' .

Definition 4. Liveness: In a kripke structure $KS = (S, R, L, I)$, liveness of KS can be expressed as follows: Given a proposition p (label of the states), a strongly connected component can not be reached from I where p is false at every state of the strongly connected component. A counterexample for the liveness of the Kripke structure requires finding such a strongly connected component.

Definition 5. In a kripke structure $KS = (S, R, L, I)$ a fair strongly connected component for a set of fairness constraints (such as a subset of state labels) is a subgraph KS' of KS where each state in $S_1 \in KS'$ is reachable from another state in KS' and each constraint is true at least one state in KS' .

Definition 6. Fairness: In a kripke structure $KS = (S, R, L, I)$, and a set of fairness condition C (subset of state labels), fairness of KS can be expressed as follows: Given a proposition p (label of the states), a fair strongly connected component can not be reached from I where p is false at every state of the fair strongly connected component. A counterexample for the fairness of the Kripke structure requires finding such a fair strongly connected component.

We describe the blind workflow verification problem as follows:

1. There are two parties: (a) the end user and (b) the agent, where the end user assigns a task $\mathbb{T}$ (represented as a string) to the agent.

2. The agent generates a kripke structure $KS = (S, R, L, I)$ and only reveals the set of states, their labels, and initial states to the end-user.
3. The end-user chooses a blind model checking problem as follows:
 - (a) The end-user can choose a state $S' \in S$ and ask the agent to present a zero-knowledge proof that S' is reachable from the initial states.
 - (b) The end-user can ask the agent to present a zero-knowledge proof on proving that the Kripke structure KG satisfies the liveness property.
 - (c) The end-user can choose a subset of state labels L' and ask the agent to prove the fairness of the Kripke structure KG for the set of constraints L' .

3. Blind model checking of Kripke structure

3.1. Overview of the solution

Given a kripke structure expressed as a directed graph $G = (V, E)$ with n nodes V and edges E . We approach to solve the reachability, liveness, and fairness of a Kripke structure by search algorithms on the Kripke structure. An example of a simple search algorithm is as follows:

1. Given the start node I , at the first iteration we find all neighbours of I via its outgoing edges. We will represent this set of neighbours as the set *Current*.
2. At the next iteration, we find all neighbours of the nodes in the the set *Current*. If these new neighbours are not previously visited, then we replace the set *Current* with these new neighbours.
3. We generate the sets *Current* for each iteration.
4. Given the sets of *Current* for all iterations we can check if certain reachability conditions for model checking the Kripke structure is satisfied or not.
5. Liveness check: We can check if we will reach a connected component by checking if nodes in the $Current_i$ for iteration i has any edge with nodes belonging to the sets $Current_0, \dots, Current_{i+1}$, i.e., *current* sets generated for iterations 0 to $i - 1$.
6. Fairness check: We can check this condition if for all such connected components have nodes that contain labels defined by the fairness condition.

Given the above procedures to check for fairness, liveness, and fairness conditions of model checking of a Kripke structure, a blind model checking problem is as follows:

1. There are two parties, a prover and the verifier. The prover knows the kripke structure $G = (V, E)$ and node labels $L(V)$. The verifier only knows the set of nodes V and node labels $L(V)$.
2. For the reachability problem, a blind model checking problem must address the following:
 - (a) The verifier asks the prover to execute the search algorithm mentioned in Algorithm 1 and report all reachable states from a given start node I .

Algorithm 1: Searching Kripke structure

Data: $G = (V, E)$ be the kripke structure, $L(V)$ be the labels to the nodes, $I \in V$ as the initial state.

Result: Reachable states from I , reachable connected components and fair connected components from I .

```

begin
  visited = (), current = (I), return_data = ()
  return_data.add(current)
  for  $i \in 1 : No\_of\_iteration$  do
    local_current = ()
    local_ret = ()
    for node in current do
      neighbours = G.neighbors(node, mode = "out")
      prove number of neighbors is correct
      if  $|neighbours| > 0$  then
        for new_node in neighbours do
          prove that edge exists
          local_ret.add(new_node)
          if new_node not in visited then
            local_current.add(new_node)
            visited.add(new_node)
        end for
      end if
    end for
    ret.add(local_ret) current = local_current
  end for
  return(ret)

```

Algorithm 2: Search for strongly connected components

Data: $G = (V, E)$ be the kripke structure, $L(V)$ be the labels to the nodes, $I \in V$ as the initial state, $\{Current_i\}$ be the set of nodes visited at each iteration, Fairness label F .

Result: Strongly connected component, fair strongly connected component

```

begin
  Cycles = ()
  Fair_cycles = ()
  for  $Current_x \in \{Current_i\}$  do
    for  $v_x \in Current_x$  do
      Find  $v_y \in Current_0 \cup \dots \cup Current_{i-1}$ 
      such that  $(v_x, v_y) \in E$ 
      Find a path  $P$  as as the set of edges from
       $v_y$  to  $v_x$  as the set of edges among the
      nodes
      Add  $P \cup \{v_x, v_y\}$  to Cycles
    end for
  end for
  for Each cycle  $\in Cycles$  do
    if Fairness condition  $\in L(v_x \in Cycle)$  then
      Add cycle in Fair_cycles
    end if
  end for
  Return(Cycles, Fair_cycles)

```

- (b) A malicious prover may not include all neighbours of a node in the set *Current* to find the next *Current* set.
 - (c) A malicious prover may include an incorrect neighbour of a node in *Current* to find the next *Current* set, i.e., it will report a wrong neighbour.
3. For the liveness problem, a blind model checking problem must address the following:
 - (a) Besides checking the concerns with the reachability of model checking, the verifier needs to check for one node in the set $Current_i$, all such states from previous iterations are considered for checking if there is an edge from this node to another node from the previous iterations.
 - (b) The prover may not reveal the cycles to the verifier, hence the verifier must be able to check if this list of cycles is correct (edges are correct) and complete (all cycles are considered).
 4. If the strongly connected components are correct and complete, then the verifier can check the fairness condition that certain node labels exist on the nodes in the cycles without the knowledge of the cycles.

3.2. Committing and hiding a kripke structure

Let $G = (V, E)$ be the Kripke structure with L be the set of node labels. The prover will generate an ordered Merkle tree as follows:

1. For each label $l_x \in L$, we generate an integer such that all labels have unique integers using the following procedure: It will create the following hash, where the node name is a name assigned to each node of the Kripke structure, Node label is the labels of the Kripke structure, and Salt is a random positive integer:

$$Hash('Node\ name' + '*' + 'Node\ Label' + '*' + Salt)$$

The Hash will be converted into an integer by converting the hexadecimal number into a decimal number. Let $\mathbb{I}(V_x)$ be the above integer representation of a node in the Kripke structure.

2. The prover will reveal the node name for each node to the verifier. The prover will reveal the integers $\mathbb{I}$ to the verifier.
3. The prover will create an ordered kripke structure with $\mathbb{I}$ where nodes are ordered according to the value in $\mathbb{I}$.
4. Let I be the index of each node created using the values in $\mathbb{I}$.
5. It will publish (send it to the verifier) the root of this Merkle tree and Hashes of the leaf nodes.

3.3. Polynomial representation of kripke structure

Let A be the $n \times n$ adjacency matrix corresponding to the edges E of the kripke structure. The prover generates a characteristic function $f(x) = \det(A - x \times I)$ where x is scalar and I is a $n \times n$ identity matrix. Let M be the adjugate

matrix of $f(x)$ where $M = \det(A) \circ A^{-1}$. The following property of the adjugate matrix can determine if there is an edge from $u \in V$ to $v \in V$:

- Let $G(x)$ be the polynomial at the (u, v) entry of the matrix M .
- If coefficient of the term x^{n-2} is 1 Gunes, Demirci, Oz and Cangul (2020).
- Otherwise it is 0.

For example, consider this matrix as the adjacency matrix of the kripke structure as follows:

$$adj_matrix = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

The adjugate matrix for this adjacency matrix is :

$$\begin{bmatrix} (x^5 - x^2) & (x^4 - x) & (x^4 + x^3) & 0 & (x^3 + x^2) & (x^2 + x) \\ 0 & (x^5 - x^2) & x^4 & 0 & x^3 & x^2 \\ 0 & 0 & x^5 & 0 & x^4 & x^3 \\ 0 & (x^4 - x) & x^3 & (x^5 - x^2) & x^2 & x \\ 0 & 0 & x^3 & 0 & x^5 & x^4 \\ 0 & 0 & x^4 & 0 & x^3 & x^5 \end{bmatrix}$$

Figure 1: Adjugate matrix

Note that according to the adjacency matrix, node 0 does not have an edge with

node 5. The $(0, 5)$ element is $f(x) = x(x + 1)$ and as the coefficient of the

x^{n-2} is zero where $n = 6$, the adjugate matrix confirms that there is no

edge from 0 to 5. The $(2, 4)$ element is $f(x) = x^4$ and as the coefficient

of the x^{n-2} is 1 where $n = 6$, the adjugate matrix confirms that there is

an edge from 2 to 4.

3.4. Converting the coefficients of the adjugate matrix into polynomials

A polynomial commitment scheme-based zero-knowledge proof for checking if the

coefficient of the term x^{n-2} is zero or not is as follows:

1. Let the set of coefficients for the polynomial $f(x)$ representing the (i, j) element of the adjugate matrix is

$$a_0, a_1, \dots, a_{n-2}, a_{n-1}, a_n.$$

2. We create a polynomial $g(x)$ that goes through these points

$$(0, a_0), (1, a_1), \dots, (n-2, a_{n-2}), (n-1, a_{n-1}), (n, a_n).$$

3. We can check the evaluation of $g(x)$ at $x = n - 2$ to determine if there is an edge from node v_i to v_j as if $g(x = n - 2) = 1$ then there is such an edge.

We can use KZG-polynomial commitment scheme to verify if such an evaluation is

correct or not. The problem with this verification is that the verifier has to

know the polynomial $g(x)$ that can reveal the edges in the Kripke structure.

3.5. Blinding adjugate matrix polynomials

In a blind polynomial evaluation problem for a polynomial $g(x)$ at $x = z$ and $g(x = z) = y$, the prover wants to prove that $g(x = z) = y$ without revealing $g(x)$ to the verifier. For the polynomial $g(x)$, we create a random scalar r and a vanishing polynomial $p(x)$ such that

$$p(x) = (x - z_1)(x - z_2) \times (x - z_k).$$

such that $\{z_i\}$ are positive integers less than n (number of nodes of the kripke structure) and $k \geq n - 2$ and there is a $k_i = n - 2$. The blinded polynomial becomes:

$$g'(x) = g(x) + r \times p(x). \quad (1)$$

This process does not allow the verifier to know the edges of the kripke structure from the adjugate matrix as we replaced $g(x)$ with $g'(x)$. This blinded adjugate matrix will be used to verify neighbours of a node as follows:

1. As shown in figure 2, let the start node for the model checking problem is V_1 and we received V_2, V_4 as the neighbours of V_1 .
2. At iteration 2, let V_3, V_6 are the neighbours of V_2, V_4 such that $V_2 \rightarrow V_3, V_4 \rightarrow V_6$.
3. The prover can claim that V_3 is neighbours of at least one node from (V_2, V_4) without revealing the exact edge $V_2 \rightarrow V_3$ as follows:
 - (a) The prover will create the polynomial $G(x) = F_{2,3}(x) + F_{4,3}(x)$.
 - (b) Next the prover will prove that $G(x)$ at $x = n - 2$ is 1.
 - (c) As $F_{2,3}(x = n - 2) = 1$ and $F_{4,3}(x = n - 2) = 0$, $G(x = n - 2)$ is 1, by proving $G(x = n - 2) = 1$ satisfies the requirement that there is at least one polynomial among $F_{2,3}, F_{4,3}$ is 1 for $x = n - 2$.

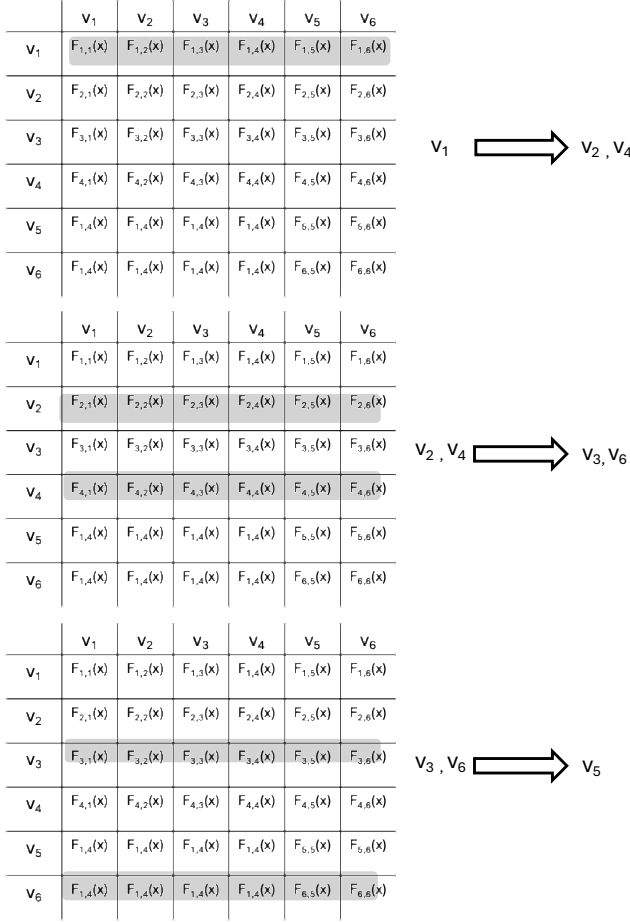


Figure 2: Example on how to hide the edgelist from iteration 2.

3.6. Incorporating Zero-knowledge proof in the searching Kripke Structure

We incorporate the zero-knowledge proofs with generating sets of reachable states per iteration from the start edges of the Kripke structure as follows:

1. For every iteration, the prover generates two sets of nodes p_neigh and n_neigh where p_neigh is the set of reachable nodes from the set of nodes in the previous iteration, and n_neigh be the set of the remaining nodes of the Kripke structure.
2. At every iteration, the prover generates a sub-matrix of the blind adjugate for rows corresponding to the nodes in the set p_neigh of the previous iteration.
3. For every new node v_x added in the set p_neigh at the current

iteration, the prover generates a KZG proof with (a) commitment to aggregated (by addition) polynomial in the sub-matrix at column i where

v_x corresponds to index i of the blind adjugate matrix.

4. Next, it generates the evaluation and the proof of evaluation of the aggregated matrix created in the previous step at $n-2$ where number of nodes of the kripke structure is n .
5. Similarly, for every node not in p_neigh i.e., it is included in the set n_neigh , it generates a KZG proof similar to the previous two steps, where evaluation of the aggregated polynomial is 0.
6. The prover sends nodes for all iterations as $\{p_neigh_i\}$, $\{n_neigh_i\}$, the KZG proofs $\{p_neigh_pf_i\}$, $\{n_neigh_pf_i\}$.
7. The verifier can use equation 7 to check if the KZG proofs are correct.

3.7. Verification of strongly connected components

The verifier will perform the following steps:

1. It will check the KZG proofs generated by the prover in the previous section.
2. It will check if there are cycles among the $\{p_neigh_i\}$ for every iteration i using algorithm 2.

3.8. Verification of unreachability of an undesriable state

Given a node label ' L_i ', and a set of nodes $\{p_neigh_i\}$ generated at the step i , the prover and the verifier will engage in the following steps to verify that $L_i \notin \{p_neigh_i\}$:

1. The prover will generate a subtree of the Merkle tree as $M' \subset M$
2. The prover will find node V_y that has the node label L_i , it will find two nodes V_x and V_z such that $\|(V_x) \leq \|(V_y) \leq \|(V_z)$.
3. The prover will prove that there is no node V_a in M' such that $\|(V_x) \leq \|(V_a) \leq \|(V_z)$.

3.9. Verification of fair strongly connected components

The verifier will receive the set of reachable states (p_neigh_i, n_neigh_i) for every iteration i and it constructs a cycle $C \subset \cup p_neigh_i$. The verifier can perform two types of checks in this cycle.

First, it can check if node with certain node label exists in the cycle C using this protocol:

1. Let the specific node label is $L_i \in L$.
2. The prover finds the node v_x with label L_i reveals the node label and salt to reveal the leaf node in the ordered merkle tree as v_x .

Algorithm 3: Searching Kripke structure

Data: M be the blind adjugate matrix for the $G = (V, E)$ be the kripke structure, $I \in V$ as the initial state, k is number of iteration.

Result: Reachable states from I per iteration

begin

```

    visited = (), current = (I), p_neigh = (), n_neigh = (),
    p_neigh_pf = (), n_neigh_pf = ()
    for  $i \in 1 : No\_of\_iteration$  do
        local_current = (),
        Local_p_neigh = (),
        Local_n_neigh = (),
        Local_p_neigh_pf = (),
        Local_n_neigh_pf = ()
        Poly1 = Combined(M(currnet:))
        for node in current do
            neigh = G.neighbors(node, mode = "out")
            if  $|neigh| > 0$  then
                for new_node in neigh do
                    Local_p_neigh.add(new_node)
                    if new_node not in visited then
                        local_current.add(new_node)
                        visited.add(new_node)
                end for
            end if
        end for
        p_neigh.add(Local_p_neigh)
        n_neigh.add(all nodes - Local_p_neigh)
        p_neigh_pf.add(p_pf(Poly1, Local_p_neigh))
        n_neigh_pf.add(n_pf(Poly1, Local_n_neigh))
        current = local_current
    end for
    return(p_neigh, n_neigh, p_neigh_pf, n_neigh_pf)

```

3. The verifier constructs the hash again to check if it matches the leaf node in the merkle tree.

Second, it can check if node with certain node label does not exists in the cycle C using this protocol:

1. Let the specific node label is $L_i \in L$.
2. The prover reveals the node label and salt to reveal the leaf node in the ordered merkle tree as v_x .
3. Next, the prover reveals two nodes v_y, v_z in C such that $\mathbb{I}(y) < \mathbb{I}(x) < \mathbb{I}(z)$.
4. The prover shows that there are paths from v_y, v_z to the root of the merkle tree.
5. The verifier checks if these paths are correct and checks if $\mathbb{I}()$ for all other nodes $v_i \in C - v_x$ are outside the range $\mathbb{I}(v_y)$ and $\mathbb{I}(v_z)$. This check ensures that v_y and v_z are correctly chosen by the prover.

Algorithm 4: Generate KZG proof for $\{p_neigh\}$

Data: Poly₁, Local_p_neigh, $n \times n$ blind adjugate matrix M

Result: KZG proof for Local_p_neigh

begin

```

    for Each  $v_x \in Local\_p\_neigh$  do
        Let  $v_x$  corresponds to  $i$ 'th column in  $M$ 
         $\{\cup(Poly_1[: i])\}$  be the addition of all
        polynomials in Poly1 for column  $i$ 
        Generate Commit( $\{\Psi(Poly_1[: i])\}$ ) from
        equation 4
        Generate Commit( $Q(\tau)$ ) from equation 5
        Genrate evaluation of  $\{\Psi(Poly_1[: i])\}$  at
         $n - 2$  as  $y$ 
    end for

```

Algorithm 5: Generate KZG proof for $\{n_neigh\}$

Data: Poly₁, Local_n_neigh, $n \times n$ blind adjugate matrix M

Result: KZG proof for Local_n_neigh

begin

```

    for Each  $v_x \in Local\_n\_neigh$  do
        Let  $v_x$  corresponds to  $i$ 'th column in  $M$ 
         $\{\cup(Poly_1[: i])\}$  be the addition of all
        polynomials in Poly1 for column  $i$ 
        Generate Commit( $\{\Psi(Poly_1[: i])\}$ ) from
        equation 4
        Generate Commit( $Q(\tau)$ ) from equation 5
        Genrate evaluation of  $\{\Psi(Poly_1[: i])\}$  at
         $n - 2$  as 0
    end for

```

4. Analysis

Theorem 1. The kripke structure search algorithm shown in algorithm 3 satisfies correctness, soundness, and zero knowledge properties.

Proof. Correctness: The correctness property of the search algorithm ensures that if there exists an edge in the kripke structure then the corresponding nodes should be included in the consecutive sets of nodes reported by the algorithm. The search algorithm is correct if the following holds:

1. Given two sets of nodes (p_neigh_i, n_neigh_i) and $(p_neigh_{i+1}, n_neigh_{i+1})$ for two consecutive steps of the search algorithm i and $i + 1$, if there is a $v_x \in V$ such that $\exists v_y \in p_neigh_i$ where $(v_y \rightarrow v_x) \in E$ for the kripke structure $G = (V, E)$ then $v_x \in p_neigh_{i+1}$.
2. $p_neigh_{i+1} \cap n_neigh_{i+1} = \emptyset$ and $p_neigh_{i+1} \cup n_neigh_{i+1} = V$.

The verifier knows the blinded adjugate matrix shown in section 3.5 and it uses KZG zero knowledge proof verification via equation 7 to check if evaluation of any equation in the blinded adjugate matrix at $n - 2$ is 1 or not. As the prover shares the blinded adjugate matrix before publishing

the sets of nodes for every iteration, it can not manipulate the blinded adjugate matrix to change the evaluation of a polynomial at $n - 2$. Hence correctness conditions 1 and 2 holds. The verifier can easily check if the 3rd condition holds as it receives the set of nodes $p_neigh_{i+1}, n_neigh_{i+1}$.

Soundness: The soundness property of the search algorithm ensures that if there does not exists an edge in the kripke structure then the corresponding noeds should not be included in the consecutive sets of nodes reported by the algorithm. The search algorithm is sound if the following holds:

1. Given two sets of nodes (p_neigh_i, n_neigh_i) and $(p_neigh_{i+1}, n_neigh_{i+1})$ for two consecutive steps of the search algorithm, for a node $v_x \in V$ such that $\nexists v_y \in p_neigh_i$ where $(v_y \rightarrow v_x) \in E$ for the kripke structure $G = (V, E)$ then $v_x \notin p_neigh_{i+1}$.
2. $p_neigh_{i+1} \cap n_neigh_{i+1} = \emptyset$ and $p_neigh_{i+1} \cup n_neigh_{i+1} = V$.

As the verifier knows the blinded adjugate matrix before the prover reveals the sets (p_neigh_i, n_neigh_i) and $(p_neigh_{i+1}, n_neigh_{i+1})$, the prover can not change the polynomial in the y 'th row and x 'th column of the blinded adjugate matrix to make its evaluation at $n - 2$ is 1. Further, using equation eq_3 it can check the such evaluation is correct. The second condition is already satisfied in the correctness property.

Zero knowledge: The search algorithm satisfies zero-knowledge property if the verifier can not know set of edges of the kripke structure $G = (V, E)$ from the sets of nodes (p_neigh_i, n_neigh_i) at every iteration i . This holds because:

1. The verifier knows the blinded adjugate matrix shown in section 3.5 where each polynomial $g(x)$ of the adjugate matrix is blinded by a random integer r and a random polynomial $p(x)$ as shown in equation 1. Note that as the verifier does not know $p(x)$ and r , it can not identify $g(x)$ and hence it can not check if $g(x = n - 2) = 1$ to identify if there is an edge from v_x to v_y where $g'(x) = g(x) + r \times p(x)$ for the element in row x and column y of the blinded adjuagte matrix. Further, $p(x)$ in equation 1 is a vanashing polynomial for $x = n - 2$ hence it does not affect actual evaluation of $g(x)$.
2. Further, at every iteration $i + 1$, for each v_x the KZG zero knowledge proof is calculated using aggregated using all polynomials in the set p_neigh_i by adding all polynomails for nodes p_neigh_i . This hides the exact node $v_y \in p_neigh_i$ that has an edge to v_y .

□

Theorem 2. *The model checking protocol for verifying unreachability to an undesirable state satisfies the correctness, zero-knowledge, and soundness property.*

Proof. Correctness: The reachability condition for an undesirable state of model checking a kripke structure checks if a node with a specific label can not be reached from the start

node. The following conditions should hold for correctness this checking:

1. Given the sets of nodes (p_neigh_i, n_neigh_i) for every iteration i , if the undesirable state v_x is actualall not reachable then it should not be in the set $\cup p_neigh_i$.

This condition holds because:

1. The prover commits to the node labels as a ordered merkle tree as shown in section 3.2 and it sends the root, leaf nodes, and node names to the verifier before revealing the sets (p_neigh_i, n_neigh_i) for all iterations.
2. Note that, given this merkle tree, the verifier checks if such undesirable state $v_x \notin \cup p_neigh_i$ using the protocol mentioned in section 3.8 where :
 - (a) the prover reveals the leaf node corresponding to v_x by revealing corresponding node label and salt. The verifier can reconstruct the hash and check it with the leaf nodes of the merkle tree.
 - (b) The verifier also calcuates the integer representation $\mathbb{I}$ for v_x using the above information and check if the published index in section 3.2 is correct.
 - (c) Next, the prover finds two nodes v_y, v_z in $\cup p_neigh_i$ such that $\mathbb{I}(v_y) < \mathbb{I}(v_x) < \mathbb{I}(v_z)$. Then the prover proves that v_y and v_z 's integer representation is correct by revealing the corresponding node label and salt information. Also, it proves correctness by proving that paths from v_y, v_x, v_z to the root of the merkle tree is correct. The prover can not manipulate these proofs because it has revealed the merkle tree developed in section 3.2 before revealing the results from the search algoritmh (algorihtm 3).
3. Further, the prover can not misreport the sets (p_neigh_i, n_neigh_i) for each iteration i as theorem 1 ensures correctness and soundness of search algorithm.

Soundness: The reachability of an undesirable state is sound if the follwing condition holds:

1. Given an undesirable state v_x , if v_x is acutaly not reachable from the start node of the kripke structure then v_x should be in this set $\cup n_neigh_i$.

This condition holds because: The prover commits to the node labels as a ordered merkle tree as shown in section 3.2 and it sends the root, leaf nodes, and node names to the verifier before revealing the sets (n_neigh_i) for all iterations.

Note that, given this merkle tree, the verifier checks if such undesirable state $v_x \in \cup n_neigh_i$ using the protocol mentioned in section 3.8 where :

1. the prover reveals the leaf node corresponding to v_x by revealing corresponding node label and salt. The verifier can reconstruct the hash and check it with the leaf nodes of the merkle tree.

2. The prover can not misreport the sets (n_neigh_i) for each iteration i as theorem 1 ensures correctness and soundness of search algorithm.

Zero-Knowledge: This property ensures that the verifier does not indeify the edges of the kripke structure from the reachability proof for an undesriable state as shown in section 3.8. This condition holds because:

- Note that, in theorem 1 we have proved zero-knowledge property of the serach algorithm for the revealed set of nodes (p_neigh_i, n_neigh_i) for all iterations.
- As shown in section 3.8 the prover presents an unreachability proof using the ordered merkle tree developed in section 3.2. In this proof, the prover only reveals (a) node label of the undesiable state and its path to the root node of the merkle tree, and (b) the same for two adjacent nodes v_y and v_z such that $\mathbb{I}(v_y) < \mathbb{I}(v_x) < \mathbb{I}(v_z)$.

□

Theorem 3. *The model checking protocol for verifying the liveness satisfies the correctness, zero-knowledge, and soundness property.*

Proof. Correctness: The model checking for liveness of a kripke structure requires to check if there is a strongly connected component among all reachable states from the start node of the kripke structure. The correctness propoerty of this check should ensure that : if there is actually such a cycle then it can be identified by the verifier. This propery holds because:

1. The prover can not misreport the sets of nodes (p_neigh_i, n_neigh_i) for all iterations of the search algorithm as theorem 1 is correct.
2. If the reported sets of nodes are correct then using algorithm shown in section 3.7 the verifier can identify if there is a cycle among the nodes in (p_neigh_i, n_neigh_i) .

Soundness: The soundness propoerty of this check should ensure that : if there is actually no such a cycle then the prover misreport the reachbale states that includes such a cycle. This property holds because: the search algorithm that generates the sets (p_neigh_i, n_neigh_i) is sound.

Zero-Knowledge: The model checking protocol for liveness check satisfies the zero-knowlege property if the verifier can not know the set of edges of the kripke structure. This condition holds because the verifier only uses the information in the sets (p_neigh_i, n_neigh_i) to find a cycle among them. As shown in theorem 1, it is not possible to identify the set of edges of the kripke structure from (p_neigh_i, n_neigh_i) . Hence the model checking protocol for verifying zero knowledge property holds.

□

Theorem 4. *The model checking protocol for verifying the fairness satisfies the correctness, zero-knowledge, and soundness property.*

Proof. The model checking of the fairness property requires checking if there is a cycle among the reachable states from the start node of a kripke structure and there is a node in this cycle that has a specific node label.

Correctness: The correctness property of this problem ensures that a prover can only include such a node in the cycle among the reachable nodes if that node actually has edges in the kripke strcuture to remain in the cycle. This property holds because:

1. This property holds beacuse of the correctness propoerty of theorem 1 where any node in p_neigh_{i+1} has at least one edge with any node in p_neigh_i .
2. As the reported sets of nodes in the serach algorithm 3 is correct, the fairness condition is correct as the prover can not include a node with specific node label to p_neigh_{i+1} without edges from the nodes p_neigh_i .
3. As shown in section 3.9 (first check), the prover reveals the node label and salt to the verifier to check if an appropriate node for an undesireable node label exists in the cycle. As the prover committed to the order merkle tree before revealing the search results on the kripke structure, the prover can not censor such node label.

Soundness: The soundness property of this problem ensures that a prover can not include a node in the cycle such that this node actually does not has edges in the kripke structure to remain in the cycle. This property holds because:

1. As shown in section 3.9 (second check), the prover presents a proof that a vertex v_x with the undesirable label does not exist in the cycle construted by the verifier.
2. This check can not be manipulated by the prover as the ordered merkle trees and their integer representation $\mathbb{I}()$ are created before generating the search results.

Zero-Knowledge: The zero-knowledge property ensures that fair model checking protocol does not reveal the set of edges of the kripke structure. This holds because:

1. The liveness property satisfies zero-knowledge property.
2. The first check in section 3.9 only reveals node label of a node that has the undesiable node label.
3. The second check in section 3.9 only reveal node labels of three nodes.
4. Hence the fairness verification protocol satisfies zero knowledge property.

□

5. Evaluation

We used Sola, Warmuth, Schäfer, Badakhshan, Rehse and Kampik (2022) dataset on business process models. We have downloaded 20 process models represented as BPMN2.0 file format. These models vary in number nodes and edges. This dataset is expressed using BPMN2.0 process modeling language as json files.

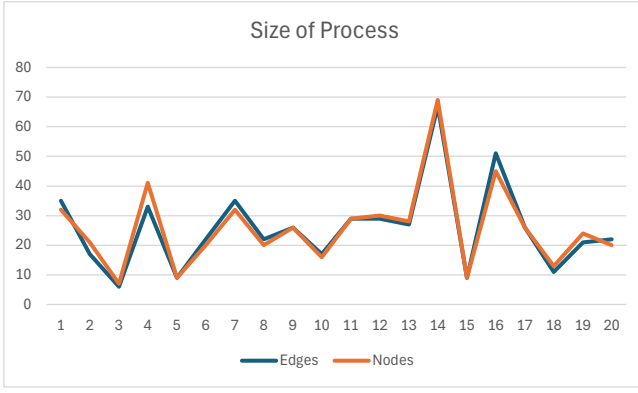


Figure 3: Process model dataset.

prompt =	Consider the following process data represented as a graph. Nodes represent task or function and an edge represent flow between two tasks where task in the 'from' node should be executed before task in 'to' node. Outcome of task execution at the 'from' node will be passed to the task in the 'to' node. The data has two dataframes. In node_data there are four fields, (a) "id" (it means an unique id for the node), (b)"label" (it means an description of the node), (c)"task_type" (it means the task that the node executes), and (d)"original_name" (means an alternate name of it or description of the task or function executed by the node). The edge_data contains three fields as (a)from_node, (b)to_node, and (c)label(may describe why these two nodes are connected). Based on this process, generate a scenario by creating a description of what each node performs and a description on why two nodes are connected. Combine all descriptions into one single paragraph. Only output this single paragraph.
Node data =	"Node_data.json"
Edge data =	"Edge_data.json"
response =	ollama.generate(model="llama3.2", prompt=prompt, stream=False)

Table 1

Prompt to generate a description of a process model.

We use this dataset to evaluate the performance of the proposed verification of process models as follows:

1. We convert these process models into a summary of process models using the prompt shown in table 1.
2. Next, we send these process description text to the agents as delegated task description and asks it to produce the process model as kripke structure, the results of the search algorithm 3, and KZG zero knowledge proofs. The agent uses prompt mentioned in table 2 to generate the kripke structure. We used "llama3.2" as the LLM model to process these prompts.

prompt =	Use the description of a process mentioned in process_description to generate a kripke structure model as a json file. A Kripke structure consists of four main parts: 1. States (S): Every possible "snapshot" or configuration the system can be in. 2. Initial States(I): Where the system starts. 3. Transitions(R): The rules or paths that show how the system moves from one state to another. 4. Labelling Function (L): This is the "dictionary." It tells you which properties (atomic propositions) are true in each specific state (e.g., in State A, is_on is true, but is_locked is false).
query =	"Process_data.json"
response =	ollama.generate(model="llama3.2", prompt=prompt, stream=False)

Table 2

Prompt to generate a Kripke structure from a process model description.

Next, we evaluate the performance of the verification of model checking as follows:

1. We have chosen 11 process models from this data sets with number of nodes and edges edges from 9 to 23.
2. First, we check the time to generate kripke structure, outcomes of the search algorithms, and KZG zero-knowledge proof generation. As shown in figure 4, we found that it takes less than 20 seconds to complete this process. But we notice that, once number of nodes exceeds 20, such execution time is increased approximately 80 seconds. This is due to the complexity of generating adjugate matrix for the kripke stucture.
3. Second, we check the time to check the KZG zero knowledge proof. As shown in figure 5, we notice that proof verification time remains constant around 5 seconds for each node included as outcome of search algorithm. This shows that verification time remains almost constant and size of the kripke structure can decide the time to check KZG zero knowledge proofs.

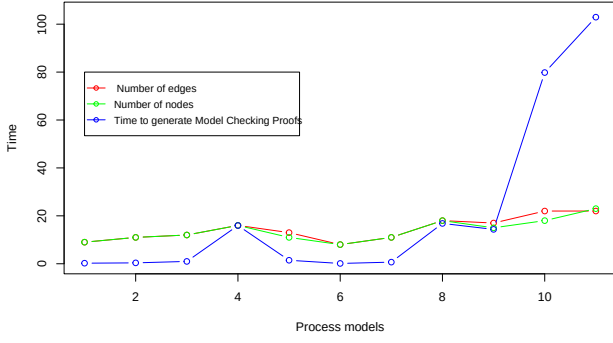


Figure 4: Adjagate matrix

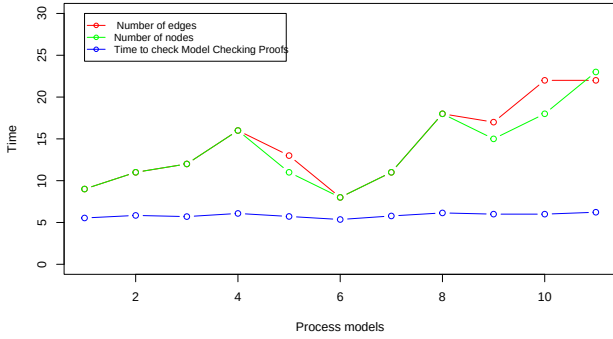


Figure 5: Adjagate matrix

6. Related Literature

In this paper, we proposed to use model checking for verify an agent's workflow and developed zero knowledge proof protocols for such a checking without the knowledge of the process models. There are various methods to secure an agent's workflow, i.e., its intentions and plans. Guardrails can be used to secure an agent's workflow where actions within the agent's workflow can be censored using rules and policies. In Huang, Hua, Zhou, Jing, Nagireddy, Padhi, Dolcetti, Xu, Chaudhury, Rawat, Nedoshivina, Chen, Sattigeri and Zhang (2025), the authors have developed guardrails for general agentic systems. In Koch (2026), the authors have explored governance standards for building agentic AI's guardrails. In Kholkar and Ahuja (2025), the authors have proposed policy as code for governing AI agents. In Jackson (2025), the authors proposed an architecture for policy as code for governing AI agents. In Muhammad, Yow and Alsenan (2026), the authors have proposed a continuous audit of the AI agent's actions against the guardrails.

Identity of an AI agent can secure its operations as authenticated agents can be trusted with its workflows. In dat

(b), the authors have described how to use digital certificate-based identity for agentic AI. In Syros, Suri, Ginesin, Nita-Rotaru and Oprea (2025), the authors have explored PKI-based governance methods for agentic AI. In Garzon, Vaziry, Kuzu, Gehrmann, Varkan, Gaballa and Küpper (2025), the authors have explored decentralised identity for agentic AI. EIP-8004 dat (c) proposed a decentralised identity management platform based on the Ethereum Blockchain, where the identity of an agent is interpreted as an NFT. EIP-8004 dat (d) also provides a reputation management service and a service to discover agents. ERC-8126 dat (e) proposed a draft standard for verifying an agent's authorization over resources and control over its wallet using zero-knowledge proof. Agent Name Service (ACS) dat (a) proposed a PKI-based identity management and discovery service for AI agents. It provides a PKI for issuing and managing a digital certificate of an agent's public key.

In this paper, we used the KZG zero-knowledge proof to verify an agent's workflow. The KGZ zero-knowledge proof protocol was developed in Kate, Zaverucha and Goldberg (2010); "Kate, Zaverucha and Goldberg (2010).

7. Conclusion

It is necessary to verify an agent's workflow to prevent undesirable outcomes due to probabilistic errors in its large language model. In this paper, we proposed to use zero knowledge proofs to verify an AI agent's workflow expressed as kripke structure. We have proved that the proposed workflow verification model satisfies correctness, soundness, and zero-knowledge properties. In future we will optimise the zero knowledge proof generation process and use it to secure code generation process by AI agents.

References

- a. Agent name service (ans) for secure al agent discovery v1.0. <https://genai.owasp.org/resource/agent-name-service-ans-for-secure-al-agent-discovery-v1-0/>. Accessed: 2026-04-03.
- b. Ai agent authentication and authorization. <https://www.ietf.org/archive/id/draft-klrc-aiagent-auth-00.html>. Accessed: 2026-04-03.
- c. Discover agents and establish trust through reputation and validation. <https://eips.ethereum.org/EIPS/eip-8004>. Accessed: 2026-04-03.
- d. Erc-8004: Trustless agent. <https://ethereum-magicians.org/t/erc-8004-trustless-agents/25098/80?page=6>. Accessed: 2026-04-03.
- e. Specialized multi-layer verification for ai agent security. <https://eips.ethereum.org/EIPS/eip-8126>. Accessed: 2026-04-03.
- Garzon, S.R., Vaziry, A., Kuzu, E.M., Gehrmann, D.E., Varkan, B., Gaballa, A., Küpper, A., 2025. Ai agents with decentralized identifiers and verifiable credentials. URL: <https://arxiv.org/abs/2511.02841>, arXiv:2511.02841.
- Gunes, A.Y., Demirci, M., Oz, M.S., Cangul, I.N., 2020. Matching number and characteristic polynomial of a graph. Journal of Taibah University for Science 14, 993–999. URL: <https://doi.org/10.1080/16583655.2020.1796275>, doi:10.1080/16583655.2020.1796275, arXiv:https://doi.org/10.1080/16583655.2020.1796275.
- Huang, Y., Hua, H., Zhou, Y., Jing, P., Nagireddy, M., Padhi, I., Dolcetti, G., Xu, Z., Chaudhury, S., Rawat, A., Nedoshivina, L., Chen, P.Y., Sattigeri, P., Zhang, X., 2025. Building a foundational guardrail for general agentic systems via synthetic data. URL: <https://arxiv.org/abs/2510.09781>, arXiv:2510.09781.

- Jackson, F., 2025. Governing autonomous ai agents with policy-as-code: A multi-layer architecture for risk, compliance, and zero-trust control. doi:http://dx.doi.org/10.2139/ssrn.5820262.
- Kate, A., Zaverucha, G.M., Goldberg, I., 2010. Constant-size commitments to polynomials and their applications, in: Abe, M. (Ed.), Advances in Cryptology - ASIACRYPT 2010, Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 177–194.
- "Kate, A., Zaverucha, G.M., Goldberg, I., 2010. "polynomial commitments". "Tech. Rep".
- Kholkar, G., Ahuja, R., 2025. The ai agent code of conduct: Automated guardrail policy-as-prompt synthesis. doi:10.48550/arXiv.2509.23994.
- Koch, C., 2026. From governance norms to enforceable controls: A layered translation method for runtime guardrails in agentic ai. URL: https://arxiv.org/abs/2604.05229, arXiv:2604.05229.
- Muhammad, A.E., Yow, K.C., Alsenan, S., 2026. Audit-as-code: a policy-as-code framework for continuous ai assurance. Frontiers in Artificial Intelligence Volume 9 - 2026. URL: https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2026.1759211, doi:10.3389/frai.2026.1759211.
- Sola, D., Warmuth, C., Schäfer, B., Badakhshan, P., Rehse, J.R., Kampik, T., 2022. Sap signavio academic models: A large process model dataset. URL: https://arxiv.org/abs/2208.12223, arXiv:2208.12223.
- Syros, G., Suri, A., Ginesin, J., Nita-Rotaru, C., Oprea, A., 2025. Saga: A security architecture for governing ai systems. URL: https://arxiv.org/abs/2504.21034, arXiv:2504.21034.

8. Appendix

In this section we will briefly discuss KZG-based zero knowledge proof protocol developed in ?. This zero-knowledge proof protocol is widely used in Ethereum-based blockchain environment ?? Our contributions are how to use KZG zero-knowledge proof to develop verification methods for RAG applications.

In this protocol, the prover has a polynomial $F(x)$ and it computes value of $F(x)$ at $x = a$ as y . It wants to convince the verifier that y is indeed the result of computing $F(x)$ for the input a . We will use the following terms:

Symbols	Meaning
$\mathbb{G}_1, \mathbb{G}_2$	Elliptic curves with bilinear pairing
$\mathbb{E}()$	Bilinear pairing function over $\mathbb{G}_1, \mathbb{G}_2$
g_1, g_2	Generators of $\mathbb{G}_1, \mathbb{G}_2$
p_1, p_2	Prime numbers for $\mathbb{G}_1, \mathbb{G}_2$
$Add(point_x, point_y), Mul(x, point_y)$	Adds elliptic curve points (generates new elliptic curve point by adding elliptic curve points $point_x$ and $point_y$) and multiplies elliptic curve points with integers (multiplies $point_y$ with positive integer x).

Setup for common reference string: During this step, the prover and the verifier computes a set of random numbers. Let G_1 be an elliptic curve for the prime number $\mathbb{P}$ and g_1 be the generator of G_1 . Let τ be a random positive integer less than $\mathbb{P}$. A set of l random numbers are generated from τ as follows:

$$p_1 = g_1, p_2 = M(\tau, g_1), p_3 = M(\tau^2(Mod(\mathbb{P})), g_1) \\ \dots,$$

$$p_l = M(\tau^{l-1}(Mod(\mathbb{P})), g_1) \quad (2)$$

where $M()$ is a function to multiply a positive integer with an elliptic curve point resulting a new elliptic curve point. We can use a ceremony to form the random number. The prover and verifier knows the above random points for both bilinear elliptic curves. We represent these random points for $\mathbb{G}_1, \mathbb{G}_2$ as follows:

$$p_1^1 = g_1, p_2^1 = M(\tau, g_1), \dots, p_l^1 = M(\tau^{l-1}(Mod(\mathbb{P}_1)), g_1) \\ p_1^2 = g_2, p_2^2 = M(\tau, g_2), \dots, p_l^2 = M(\tau^{l-1}(Mod(\mathbb{P}_2)), g_2) \quad (3)$$

Commitment to a function: Let $F(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$ be the polynomial of degree n . We can create a commitment to $F(x)$ as follows:

$$Commit(F(x)) = Add(Mul(a_0, p_1^1), Mul(a_1, p_2^1), \dots, Mul(a_n, p_n^1)) \quad (4)$$

Proof of Evaluation for a Polynomial: For a function $F(x)$, let $F(x = a)$ is y . The prover creates a proof of evaluation as a point in the elliptic curve in $\mathbb{G}_1$ as follows: Let,

$$Q(x) = \frac{F(x) - y}{x - a} \quad (5)$$

$$= r_0 + r_1x + r_2x^2 + \dots + r_nx^n$$

$$Commit(Q(\tau)) = Add(Mul(r_0, p_1^1), Mul(r_1, p_2^1), \dots, M(r_n, p_n^1)) \quad (6)$$

Verification of proof of evaluation: The prover sends the verifier the following values:

1. The commitment to the function $F(x)$ as $Commit(F(\tau))$ in $\mathbb{G}_1$.
2. The set of values of function evaluation at a as y .
3. The set of commitment for proof of function evaluation to the functions as $\{Commit(Q(\tau))\}$ in G_1 .

The verifier checks the validity of the commitment to the function evaluations by checking if the following equivalence holds:

$$\mathbb{E}(Commit(Q(\tau)), (\tau - a)) \equiv \mathbb{E}(Commit(F(\tau)) - y, g_2), \quad (7)$$

where $Commit(Q(\tau))$ and $Commit(F(\tau)) - y$ are computed using elliptic curve $\mathbb{G}_1$ and $(\tau - a)$ is computed using the elliptic curve $\mathbb{G}_2$. Note that the above equation checks if the commitment to the proof of function evaluation is correct, i.e., the polynomial division for equation 5 has no remainder. Note that verification requires multiplication of two elliptic curve points and hence we used bilinear pairing-based elliptic curves for this purpose.